# From Voice to Shell: A SLM-Based Assistant for IoT Maintenance Tasks on the Edge

Ángela Gijón Flores, Cristina Bolaños Peño, Henry Llumiguano Solano, Jesús Fernández-Bermejo Ruiz, Félix Jesús Villanueva Molina, and Fernando Rincón Calle, *Member, IEEE*

***Abstract*—Internet of Things (IoT) deployment relies on complex command-line interfaces that handle multiple commands, difficult to master, especially for nonexpert or novel technicians and maintainers. Spoken dialog arises as the most natural interaction for providing assistance to humans. Therefore, this study presents a privacy-preserving multilingual voice assistant, completely run on edge devices, that transforms natural language instructions into executable commands for heterogeneous IoT deployments. Two contexts are considered: Docker commands, whose options and arguments are publicly accessible, and an emergency luminaires system commands, a proprietary deployment whose information is not previously available to pretrained models. Multiple datasets for both contexts have been drafted to fine-tune and evaluate proposed models. A benchmark and its results for comparing proposed models are exposed, as well as a functional implementation of the voice assistant, composed of models from the comparison. The proposed system achieves a 3.15 word error rate (WER) and 98.57% semantic similarity for transcribing voice data, and takes only 2.8 s to generate a rightful command with 81.36% accuracy and not needing GPU acceleration, demonstrating reasonable performance on tested edge devices. This study's findings show that carefully fine-tuned small language models (SLMs) deliver high efficiency for speech-driven IoT device management at a fraction of the computational cost, eliminating cloud dependence and protecting sensitive audio data.**

***Index Terms*—Assistant, command generation, edge, Internet of Things (IoT), small language model (SLM), speech to text (STT).**

## I. Introduction

MAINTENANCE and administration of information technology (IT) infrastructures still rely heavily on command-line interfaces. In complex environments such as industrial plants, Internet of Things (IoT) deployments, and smart city systems, this interaction type offers fine-grained precision and control, although it requires operators to master and remember multiple platform-specific commands across different operating systems and tools. This results in an expensive and time-consuming training process, coupled with an imperative need for continuous upskilling [1].

To streamline command-line interaction and cope with the exponential growth of connected devices, researchers aim to find a solution by employing natural language processing (NLP) techniques. Large language models (LLMs), such as DeepSeek [2] and Llama [3], can transform natural language instructions into precise commands after learning standard operational practices [4]. However, their implementation is not without drawbacks. As exposed in a recent survey on LLMs security and privacy [5], hosting these models in cloud environments increases latency, demands continuous connectivity, and forces reliance on proprietary APIs, while also increasing costs and posing data-confidentiality risks. Moreover, inference workloads require substantial computational power, leading to a large environmental footprint and reliance on advanced and expensive infrastructure. As an alternative, small language models (SLMs) are further explored, as they keep a reasonable generative capacity while holding a lower number of parameters, hence operating at a reduced computational cost [6]. SLMs could be executed directly on the edge, safeguarding data confidentiality and avoiding reliance on external infrastructure, whenever a memory footprint and inference time-tailored optimization is conducted.

Beyond command and deployment environment knowledge, IoT maintenance crews may need assistance to recall several commands and tools [1]. Spoken dialog is the most natural interaction to assist humans [7], thus voice data processing is required. Speech to text (STT) models, such as Whisper [8] and Wav2Vec2 [9], can transform voice instructions into transcript text, being able to be processed by the previously mentioned generative models. Some of these models could be embedded on an edge device, enabling privacy preservation and safeguarding of voice data, considered as *biometric data*, in compliance with the General Data Protection Regulation (GDPR) by European Union (EU).[1] Despite their state-of-the-art results, most open-source STT models are strongly biased by language or spoken accent, as they have been mainly trained on standard English corpora in a nonsupervised manner [10]. Therefore, their performance on transcribing audio data from nonnative speakers or nonstandard accents has to be further evaluated, such as their ability to run

Received 17 July 2025; revised 10 October 2025; accepted 7 November 2025. Date of publication 13 November 2025; date of current version 8 January 2026. This work was supported in part by MICIU/AEI/10.13039/501100011033 and European Union (EU) "Next Generation EU" PRTR under Grant TALENT TO CONCEPT (PDC2023-145865-C32); in part by MICIU/AEI/10.13039/501100011033 and EU FEDER funding under Grant OASIS (PID2023-148285OB-C41); in part by HORIZON EU Program under Agreement 101213323 (MAYA Project); and in part by the 2023-UNIVERS-11977-FPU08 University Faculty Training Program (FPU) with European Social Fund Plus (ESF+). *(Corresponding author: Cristina Bolaños Peño.)*

This work involved human subjects or animals in its research. Approval of all ethical and experimental procedures and protocols was granted by the Social Research Ethics Committee (SREC) of the University of Castilla–La Mancha (UCLM) under Application No. CEIS-2025-119450.

The authors are with the University of Castilla–La Mancha, 13071 Ciudad Real, Spain (e-mail: Cristina.Bolanos@uclm.es).

Digital Object Identifier 10.1109/JIOT.2025.3632638

[1] https://eur-lex.europa.eu/eli/reg/2016/679/oj

2327-4662 © 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

on resource-constrained devices. These models can be used as an interface for *agentic systems*, in which an artificial intelligent (AI) *agent*, given a textual input, is capable of reasoning, employing tools, and producing precise responses along with user and environment contexts [11]. Furthermore, AI agents based on SLMs can reduce latency, power consumption, and costs, processing data on the device, while holding most of the reasoning capabilities [12]. This agentic approach, along with a speech recognition interface, should be suitable for an IoT environment assistant deployed on the edge.

Regarding available commands, many IoT environments are configured and orchestrated using Docker[2] service [13], which enables running microservices on edge devices to control and monitor sensors and actuators. Simultaneously, some IoT deployments require a high level of customisation to optimize communications and data processing, so tailored proprietary command-line interfaces and tools are designed and employed. As an example, the present work describes a customized IoT deployment for emergency luminaires in a building where the need for a domain-specific maintenance assistant has arisen. Emergency lighting systems are essential for occupant safety, as they provide illumination during power outages or emergencies to guide occupants toward exits, facilitating a safe evacuation. Despite being already trained on large corpora, generic pretrained language models perform poorly at generating accurate commands, as further demonstrated in this study. Moreover, model performance decreases even more when trying to generate proprietary commands, as they are not publicly documented or widely available. Language models, while powerful, are commonly trained on open-source or publicly accessible corpora, thus lacking knowledge of domain-specific syntax and semantics required to operate effectively in private, industrial, or customized environments. This highlights the importance of fine-tuning with small domain-specific datasets to bridge the gap between general-purpose language capabilities and the specialized knowledge required in proprietary system contexts [14].

This study focuses on the following research question: *How effective are fine-tuned SLMs combined with embedded STT models in generating accurate commands for IoT infrastructure maintenance by nonnative English speakers in offline, resource-constrained environments?* The feasibility of a voice assistant for IoT deployments, hosted mainly on edge devices, is evaluated in terms of precision and efficiency, demonstrating reliable generation capabilities.

Therefore, this work makes the following contributions.

1) A privacy-preserving multilingual voice assistant that generates executable command-line operations related to IoT maintenance environments from a voice instruction. Unlike cloud-based LLM solutions, the proposed system is capable of running on the edge with high performance, safeguarding both user and IoT deployments' sensitive data, reducing latency, and operating within common computing limitations of IoT environments and portable maintenance laptops.
2) An end-to-end feasibility demonstration of the proposed voice assistant in real IoT maintenance workflows while analyzing the inherent challenges of *audio* → *text* → *action* pipelines: inference time, memory, and CPU/GPU usage across model sizes and architectures. This use case is tested on edge devices, generating commands in ~2.8 s with ~81% accuracy on CPU-only devices, increasing up to ~85% in a GPU is available, and more powerful models could be loaded. Robustness to accent and language variability is strengthened by a lightweight, domain-trained spell-checking stage, which increases accuracy by >25% without significantly increasing latency.
3) A systematic benchmark of state-of-the-art STT, *text to text*, and *text generation* models to perform the *audio* → *text* → *action* pipeline under realistic conditions: nonnative English speakers, domain-specific commands, and resource-constrained devices. It is supported by manually drafted datasets, containing Docker and emergency luminaires system commands, designed within this study and accessible at [15]. The evaluation yields that fine-tuned *text to text* SLMs can achieve over 90% exact command matches while remaining light enough for CPU-only edge execution.

Together, these results yield that domain-specific fine-tuned *text to text* SLMs and compact open-source STT models can be combined into practical, secure, and cost-efficient voice assistants for IoT maintenance on the edge, enabling privacy-preserving and cloud-independent support in industrial and IoT environments.

The remainder of this study starts with the background and related work, outlined in Section II. In Section III the followed method is presented: system overview (Section III-A), model selection (Section III-B), data (Section III-C), training (Section III-D), and evaluation (Section III-E) of proposed models. Then, this work is described in Section IV. Finally, conclusions and future work are expressed in Section V.

## II. Background

In recent years, NLP techniques have represented a qualitative leap in both research and application of AI [16]. Among the different deep learning (DL) methods for solving NLP tasks, the *transformer* architecture [17] rises as a groundbreaking alternative by applying self-attention to process entire sentences simultaneously rather than sequentially, enabling obtaining state-of-the-art results. The popularity of this kind of model has been increased by the appearance of several libraries, which enable an easy setup for inference and training and offer access to many pretrained models based on the mentioned architecture.

Nowadays, there is a trend toward building larger models, commonly called LLMs, targeting precision and accuracy of results.[3] Although they are highly promising, LLMs require a vast amount of memory and computational resources, not

[2]https://www.docker.com

[3]https://lambda.ai/llm-benchmarks-leaderboard

being well-suited for embedded systems or edge devices [18]. In this context, SLMs rise as cost-effective alternatives while having less number of parameters (roughly from 100 M to 5 B) [12], [19]. They have reached similar accuracy results compared to LLMs in reasoning and arithmetic tasks while requiring only a fraction of the computational cost and offering inference times compatible with resource-constrained devices. Following this same trend toward increasingly compact architectures, recent proposals such as super tiny language models (STLMs) [20] have emerged. An example is TinyStories [21], which has demonstrated that a model with a single transformer block and around 1-M parameters is capable of generating coherent narratives in a specific domain. These proposals show a growing interest in ultrasmall models to improve efficiency and enable their deployment in environments with limited resources. However, data gathering and curation for training and evaluation are critical steps when employing SLMs or downsized alternatives, as many performance errors originate from data quality issues [22]. This trend change is also affected by the gradual shift from cloud to edge computing driven by privacy, latency, and sustainability requirements [23].

Data privacy protection is even more important when voice data has to be stored or processed, as it is considered *biometric data* by the GDPR. The voice assistant of this study is required to ensure precision under real conditions and protection of user voice data, while not depending on cloud infrastructures and reducing latency, so user engagement is not affected [24]. Whisper [8] from OpenAI is an open-source STT model that competently transcribes and translates multilingual audio data into text, obtaining its Medium variant 7.5 and 6.35 word error rate (WER) for transcribing English and Spanish audio data, respectively, and 21.4% BLEU score for translating Spanish audio data into English text. WER [25] measures how accurate text predictions are compared to their ground truths, employing the number of insertions, deletions, and substitutions. BLEU [26] measures similarity between a predicted text and one or more reference translations. Wav2Vec2 [9] and Conformer [27] architectures, that appeared before Whisper, are quicker and less complex approaches for STT which obtain ∼8 and ∼7 WER for transcribing English audio data, respectively. Some variants of these architectures could be run on an edge device; therefore, this study evaluates them in terms of performance and robustness to be used as the STT required for a voice assistant. It is essential to address its translation capabilities in order to be deployed in a multilingual production environment, as assistants are likely to achieve higher WER when processing non-English audio data [28]. To mitigate potential language-induced errors, previous studies suggest *fix* or *correct* obtained texts from STT models to reduce hallucinations and WER, even in low-resource languages [29].

There is a growing trend toward automating IoT deployments through the application of NLP techniques. EcoMate [30] integrates GPT-3.5[4] with Home Assistant[5] to transform a given input into YAML routines ready for execution, achieving an accuracy of about 70%, thus simplifying the interaction with home automation systems. RuleBot++ [31] uses GPT-4 Turbo from OpenAI API[6] to generate "trigger–action" rules in Home Assistant and validates the syntax through external calls, thus streamlining home automation at the cost of cloud dependence and occasional hallucinations. For Linux command-line solutions, NL2SH-ALFA benchmark [32] evaluates the accuracy of language models for transforming natural language instructions into Bash commands, although it does not address real-time integration. Generative IoT (GIoT) [33] demonstrates that self-hosted LLMs can coordinate a swarm of devices by generating "trigger–action" rules and translating them into MQTT commands with an accuracy of 79.5%. LLMind [34] shows how natural language instructions are converted into executable schedules to coordinate multiple IoT devices and AI modules without manual intervention, targeting complex domestic or industrial environments. These approaches address the *text* → *action* challenge in IoT environments and demonstrate the potential of LLMs for automating IoT tasks, but they also highlight the limits of cloud-based solutions and LLMs' general-purpose nature, reinforcing the relevance of adopting a local, domain-trained, and specificity-oriented approach such as the one proposed in this study.

Mastering command-line syntax involves a high cognitive load and effective mechanisms to reduce human error and learning time [1]. In order to address these issues, several research studies focus on employing NLP techniques to assist users with command-line interfaces. AST-Transformer [35] links user requests with relevant fragments of the user manual (`man` Linux command), facilitating the explanation of commonly used Bash commands and "teaching" rather than automatically executing. Project CLAI [36] encapsulates a set of tools (generic Bash command correction, example retrieval, and documentation) to provide suggestions to the user without exiting the shell. Segmented invocation transformer (SIT) [37] optimizes Bash command generation by incorporating constituency trees.

Few literature studies have completed the complete *audio* → *text* → *action* pipeline. A recent framework for 5G networks [38] combines cognitive machine reasoning with NLP, which processes natural language instructions, potentially dictated by voice, and transforms them into YAML/JSON descriptors that an orchestrator can deploy without human intervention. Although it is applied to the telecommunications industry, the subject of this study it reduce service provisioning time and improve network metrics. However, it requires powerful hardware and constant connectivity, making it feasible to be deployed in embedded IoT gateways. Related to this study domain, AssistantGPT [39] is capable of translating spoken or written commands into OS calls over 90% success rate. Nonetheless, it is entirely dependent on the OpenAI API, which entails continuous connectivity, exposure to sensitive data, variable costs, and no awareness of proprietary commands. Regarding studies that *seem* to include an *agentic* workflow, AutoGen [40] holds several *agents* acting as classifiers, deciding which agent should deliver the user request, rather than directly executing the requested action. HERA [41]

[4]https://platform.openai.com/docs/models/gpt-3.5-turbo

[5]https://www.home-assistant.io

[6]https://openai.com/index/chatgpt

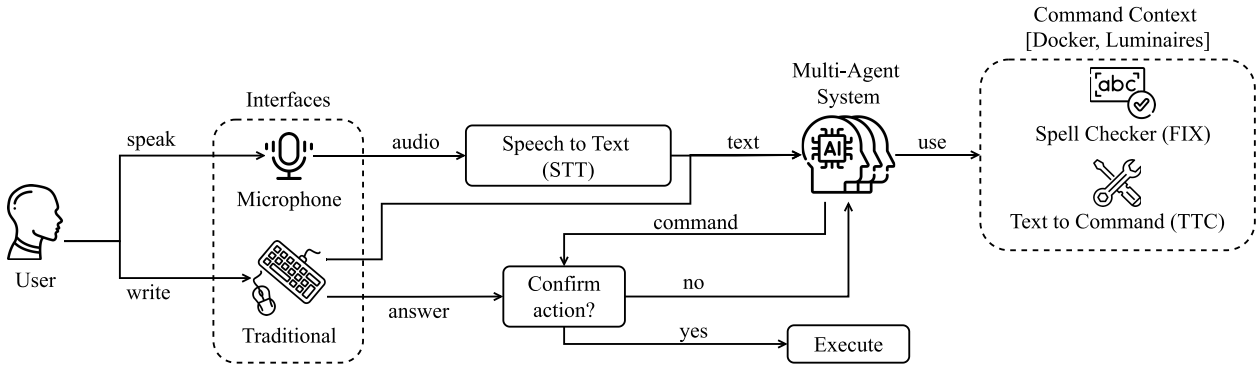



Fig. 1. Proposed system. This study contemplates the evaluation and selection of STT, FIX, and TTC components. All icons used were obtained from flaticon.com.

distributes subtasks between an SLM (edge) and an LLM (cloud) to maximize performance, acting as a classifier that only redirects queries. RouteLLM [42] is a lightweight *router* which decides whether to send user requests to either an LLM or an SLM without human interaction. To the best of our knowledge, there is no solution that implements a completely implemented agent, controlled by voice, executed locally, capable of holding a conversation with the user while reasoning and employing tools, which could be specialized models or other agents, or being based only on SLMs [12].

As a lightweight alternative to the mentioned approaches, this study proposes fine-tuning SLMs on either public or proprietary commands, Docker, and an emergency luminaires system, respectively, and composing a locally deployed multilingual voice assistant for command generation from natural language instructions, completely autonomous and not needing cloud-based models or external APIs. It combines on-device execution, supervised training with tailored data, and human validation of the output before execution, therefore consolidating a feasible voice assistant in closed, resource-constrained IoT environments.

## III. Methods

### A. System Overview

This study proposes a privacy-preserving voice assistant pipeline to be run on edge devices and used by IoT deployment maintenance staff. The workflow of the proposed system is shown in Fig. 1. Instructions are captured by the device's embedded microphone in the form of audio data. Traditional input devices, as keyboards, are also supported as fallback interface options. Audio data is then either transcribed or translated to English, if needed, by an STT model, configured with the user's language. This setup enables multilingual functionality. Then, the obtained text from system interfaces would be processed by an agentic subsystem, although not implemented within this work, in order to extend user requests to produce high-quality prompts with deployment information and context for the following components. This study focuses on building the *audio* → *text* → *action* workflow, leaving the mentioned subsystem undetermined, thus evaluating the rest of the components in a standalone manner. As previous research suggests [29], a *spell checker* (FIX) trained on domain-specific transcripts is proposed to reduce potential hallucinations and errors produced by the STT component. Once a curated text is obtained, the text to command (TTC) component, trained on domain-specific data, produces a command ready to be executed on a terminal. However, human intervention is required to confirm the execution of the given command [43].

In order to build STT, FIX, and TTC components of the proposed system, this study.

1) Datasets comprising *(prompt, command)* samples are built to train the TTC component. As TTC is context-dependent, two datasets are formed: one for Docker (public) and the other for the emergency luminaires system (proprietary). Datasets are split into *train*, *validation*, and *test* data for training and evaluation.
2) Voice data of human subjects enunciating prompt samples from *test* splits of previously mentioned datasets is recorded to build audio datasets, so STT performance is evaluated.
3) Datasets comprising *(input, target)* samples are build to train FIX components. Data is obtained from STT errors. Datasets are split into *train*, *validation*, and *test* data for training and evaluation.
4) A benchmark for evaluating and comparing proposed models is designed employing well-known standard metrics.
5) A use case to implement this study system is proposed and conducted.

### B. Model Selection

Models used as STT, FIX, or TTC components in this study are shown in Table I. Models trained on the STT task transform audio data into transcript text. Whisper, Wav2Vec2, and conformer architectures for STT are evaluated within this study to compose the STT component. In the case of *text* → *text* transformation, there are two different approaches: *text to text*, commonly used for translation or summarization in a text-in–out fashion, and *text generation*, which produces natural, fluent, and coherent text. This study proposes T5 variants and derived, like Flan-T5, as *text to text* alternatives for the TTC component. They are widely renowned for their versatility to reformulate any NLP task into a translation problem. As for *text generation* models, this study evaluates several

TABLE I
MODELS EVALUATED IN THIS STUDY

| Model | NLP Task | Used as | Params. | Size (MB) | FLOPS[1] (GFLOPS) | Ref. | Checkpoint(s)[2] |
|---|---|---|---|---|---|---|---|
| T5 Small | *text to text* | TTC, FIX | 60.5M | 233 | 1.46 | [44] | google-t5/t5-small |
| T5 Base | *text to text* | TTC | 223M | 852 | 5.12 | [44] | google-t5/t5-base |
| T5 Large | *text to text* | TTC | 738M | 2816 | 23.16 | [44] | google-t5/t5-large |
| Flan T5 Small | *text to text* | TTC | 77M | 297 | 1.46 | [45] | google/flan-t5-small |
| Flan T5 Base | *text to text* | TTC | 248M | 948 | 5.37 | [45] | google/flan-t5-base |
| Flan T5 Large | *text to text* | TTC | 783M | 2991 | 18.05 | [45] | google/flan-t5-large |
| TinyStories 1M | *text generation* | TTC | ∼1M | 49 | 0.14 | [21] | roneneldan/TinyStories-1M |
| TinyStories 1M Instruct | *text generation* | TTC | ∼1M | 49 | 0.14 | [21] | roneneldan/TinyStories-Instruct-1M |
| Pythia 14M | *text generation* | TTC | 14M | 104 | 0.29 | [46] | EleutherAI/pythia-14m |
| GPT 2 | *text generation* | TTC | 137M | 525 | 12.12 | [47] | openai-community/gpt2 |
| GPT 2 Medium | *text generation* | TTC | 380M | 1452 | 34.67 | [47] | openai-community/gpt2-medium |
| GPT 2 Large | *text generation* | TTC | 812M | 3099 | 75.73 | [47] | openai-community/gpt2-large |
| SmolLM2 135M | *text generation* | TTC | 135M | 260 | 4.60 | [48] | HuggingFaceTB/SmolLM2-135M-Instruct |
| SmolLM2 360M | *text generation* | TTC | 360M | 693 | 12.90 | [48] | HuggingFaceTB/SmolLM2-360M-Instruct |
| CodeGen Mono | *text generation* | TTC | 350M | 764 | 164.85 | [49] | Salesforce/codegen-350M-mono |
| Qwen3 | *text generation* | TTC | 752M | 1449 | 312.35 | [50] | Qwen/Qwen3-0.6B |
| DeepSeek Coder Instruct | *text generation* | TTC | 1.35B | 2570 | 682.43 | [2] | deepseek-ai/deepseek-coder-1.3b-instruct |
| Llama 3.2 Instruct | *text generation* | TTC | 1.24B | 2366 | 650.34 | [3] | meta-llama/Llama-3.2-1B-Instruct |
| Whisper Tiny | *speech to text* | STT | 37.8M | 148 | 25.67 | [8] | openai/whisper-tiny |
| Whisper Base | *speech to text* | STT | 72.6M | 281 | 67.53 | [8] | openai/whisper-base |
| Whisper Small | *speech to text* | STT | 242M | 926 | 301.29 | [8] | openai/whisper-small |
| Whisper Medium | *speech to text* | STT | 764M | 2918 | 1067.68 | [8] | openai/whisper-medium |
| Wav2Vec2 Base | *speech to text* | STT | 94.4M | 360 | 21.51 | [9] | facebook/wav2vec2-base-960h<br>facebook/wav2vec2-base-10k-voxpopuli-ft-es |
| Conformer Large | *speech to text* | STT | 120M | 431 | 20.35 | [27] | nvidia/stt_en_conformer_ctc_large<br>nvidia/stt_es_conformer_ctc_large |
| OPUS MT ES-EN | *translation* | *STT*[3] | ∼74M | 299 | 11.37 | [51] | Helsinki-NLP/opus-mt-es-en |

[1] Estimated using https://docs.pytorch.org/docs/stable/profiler.html.
[2] From HuggingFace Hub https://huggingface.co/models.
[3] Supports translation for pure STT models which do not provide it, like Wav2Vec2 and Conformer architectures.

alternatives. TinyStories models are compact alternatives trained with synthetic data to assess narrative coherence in specific domains. Pythia is a suite of autoregressive models with standardized training and reproducible scaling, available in very small sizes. GPT2 family's autoregressive nature provides a valuable baseline. SmolLM2 compact instruction-tuned models are designed for low-capacity environments. Qwen3 is a general-purpose model tuned for various instruction-following tasks. CodeGen Mono is pretrained on monolingual Python code, yielding high performance in shell-command generation. As larger alternatives, DeepSeek Coder Instruct and Llama 3.2 Instruct are proposed to compare SLMs with rather than using them for the TTC component, testing if they yield better results than SLMs if trained on domain-specific data.

Therefore, this study evaluates all *text to text* and *text generation* models for being the TTC component of the proposed system, and STT models for being the STT component. For the FIX component, T5 Small is evaluated for performing fast transcript corrections, as it is the smallest proposed *text to text* model, and due to the outstanding performance of this type of model, further shown in this study.

### C. Data

*1) TTC Datasets:* Related research highlights the importance of fine-tuning with small, balanced, high-quality, domain-specific datasets to effectively obtain accurate proprietary commands and reducing hallucinations and incorrect predictions [14], [22], [52]. Thus, two hyper-specialized *text to text* datasets, for Docker and the emergency luminaires system, are built within this study.

To build the Docker dataset, broadly used for IoT deployments on the edge [13], 133 relevant command options have been manually selected (i.e., `image`, `container`, `volume`, etc.) and 24 prompt samples for each one have been manually drafted, yielding a total of 3192 *(prompt, command)* samples. Each sample has been created with reference to the official user manual and tested in an isolated environment to check that the syntax and behavior are correct. Some precautions have been taken in order to reduce potential generation errors: redundant aliases removal (i.e., `docker images` and `docker image ls`), and concrete identifiers usage instead of generic placeholders (i.e., `python` instead of `image`). The outcome is a compact, consistent, and functionally robust corpus, manually verified, that provides a solid foundation for reliable specialization of lightweight models in real container administration tasks. As an example, the prompt "show how much disk space the Ubuntu container is using" is paired with the command `docker inspect -s ubuntu`.

As a proprietary service, emergency luminaires system commands present a challenge to previous strategies. No language model has ever been trained on this particular set of commands or any information about this specific deployment. These commands have to be memorized entirely by technicians, being an impractical and costly task that would considerably lengthen common diagnostic and maintenance operations. In practice, the command-line is accessed by two distinct roles: installers and maintainers. Installers are the professionals responsible for the initial deployment of the system, while maintainers

TABLE II
SAMPLES OF EMERGENCY LUMINAIRES SYSTEM COMMANDS

| Category | Prompt | Command |
|---|---|---|
| Luminaire | Set the screen brightness of luminaire seventy to 90 | set_brightness 70 90 |
| Group | Create a group named sensor with luminaires 4 and 33 | add_group sensor 4,33 |
| Tests | Show me the complete tests results log | get_test_results |
| Augmented security | Turn augmented security on for luminaire 1600 | augmented_on 1600 |
| Sink | Apply configuration to all sinks | config_sink |
| DBR[1]mode | Deactivate DBR for luminaire with number ID 578 | dbr_off 578 |
| Beacons | Turn on Bluetooth beacon mode on luminaire fifteen | enable_beacons 15 |
| Provisioning | Deactivate provisioning in Wirepas configuration | disable_provisioning |
| Diagnostic status | Show me each luminaire diagnostics status report | get_diagnostics |
| Mesh graph | Show me a connectivity map of the lighting system | get_mesh |
| Emergency mode | Activate emergency mode on luminaire with ID 1740 | on 1740 |
| GPIO Output | Activate second output for device with code 666 | set_gpio_output 666 10 |
| Pictogram | Place the down arrow pictogram on luminaire sixty one | set_pictogram 61 1 |
| Help/Exit | How do I stop a functional test on a luminaire? | help stop_functional_test |

[1] *Demand-Based Regulation* mode. It dims the lighting when energy demand is high.

ensure its continuous, safe, and efficient operation after the completion of the deployment. The latter often have less in-depth knowledge of the underlying command-line than the development team, so having a system capable of generating and validating such commands would reduce time and risk of manual error. This command-line functions include adding, removing, and grouping luminaires, and also consulting and modifying their parameters, such as brightness and operation mode (normal, provisioning, discovering, augmented security, or diagnostic). Moreover, functional and autonomy tests can be run on luminaires to ensure safety and proper operation. Each luminaire operates autonomously within a dynamic homogenous mesh of connected devices, in which one of them acts as the mesh controller (sink). Through the developed command-line, mesh status can be monitored, and maintenance operations can be performed. With the guidance from the system developers, 53 different commands have been gathered covering the full range of maintenance and management tasks. For each command, 24 prompt samples have been manually drafted, yielding a total of 1272 samples. Table II shows some representative examples. Every prompt has been reviewed by the system developers themselves, obtaining a verified and diverse corpus, not including any company private data or credentials, and perfectly aligned with the real operating procedures of the lighting system.

*2) Voice Data:* In order to evaluate STT capabilities of the system and instruction comprehension, a realistic audio dataset is built. Prompts from *test* splits of gathered TTC Docker and luminaires datasets (see Section III-C1) have been used to be enunciated by technicians and recorded. On one hand, the Docker audio dataset comprises 3192 audio files, 3.92 h, recorded from 12 different subjects (four women and eight men). Read prompts are in English, although all subjects are nonnative English speakers. This setup would demonstrate STT robustness to transcribe speech data. On the other hand, the luminaires audio dataset holds 1060 audio files, 1.18 h, recorded from five different subjects (two women and three men). In this case, prompts are recorded in both English and Spanish, enabling highlighting differences between the transcribe and translate processes made by the STT component.

Being this data was gathered from human subjects, and considered as a *biometric data* by the GDPR, a strict ethical protocol has been followed, which has been approved by the Social Research Ethics Committee (SREC) of the University of Castilla–La Mancha (UCLM) under reference number CEIS-2025-119450. Human participation is justified, and the benefits and risks have been adequately assessed for participants, developing a participation mechanism that warrants equal opportunities for research collaboration. Every participant has been provided an informed consent document including the necessary information about the research purposes and requirements. It also complies with the regulations in force regarding personal data protection.

*3) Spell Checker:* Transcript errors are needed to train the FIX component so it can fix obtained texts from the STT component in production, and the TTC component would receive rightful inputs in English. Two different datasets are drafted to fulfill this purpose, one per command context (Docker and luminaires). Audio data gathered at Section III-C2 has been processed by all STT models proposed at Section III-B. Obtained transcripts are normalized (lowercased, symbols and diacritics removed, leading and trailing spaces removed) and filtered (only English transcripts). There have been found 44 hallucinations (18 from Whisper Tiny, 11 from Whisper Base, 8 from Conformer Large, and 7 from Wav2Vec2 Base), 2 from transcribing English audio files, and 42 from translating Spanish audio data into English text, in which one or more words are repeated more than 7 times. After removing found hallucinations and transcript duplicates, the datasets have been formed: a dataset for Docker command context with 10 681 balanced samples (8544 for *train*, 1068 for *validation*, and 1069 for *test*), and the analog for luminaires command context comprising 4347 balanced samples (3477 for *train*, 435 for *validation*, and 435 for *test*). This study uses these datasets to train T5 Small to conform to the FIX component of the proposed system.

### D. Training

In this study, 20 *transformer* models are trained: 6 *text to text* and 12 *text generation* models for the TTC component, and 1 *text to text* model for the FIX component. Custom datasets exposed in Section III-C have been used for this purpose. STT models from Section III-B have not been fine-tuned

TABLE III
HYPERPARAMETERS OBTAINED WITH OPTUNA LIBRARY FOR BEST PERFORMANCE

| Dataset | Model | Task | Learnable parameters | Epochs | Warmup steps | Batch | Gradient accumulation steps | Input length | Learning rate | Learning metric | Inference target length |
|---|---|---|---|---|---|---|---|---|---|---|---|
| *Docker* | T5 Small | TTC | 60,506,624 | 19 | 128 | 16 | 2 | 128 | 3.163e-04 | *loss* | 128 |
| | T5 Base | TTC | 222,903,552 | 16 | 92 | 16 | 1 | 128 | 1.994e-04 | *loss* | 128 |
| | T5 Large | TTC | 737,668,096 | 15 | 50 | 4 | 4 | 128 | 1e-04 | *loss* | 128 |
| | Flan T5 Small | TTC | 76,961,152 | 19 | 161 | 16 | 2 | 128 | 2.494e-04 | *loss* | 128 |
| | Flan T5 Base | TTC | 247,577,856 | 18 | 136 | 16 | 1 | 128 | 1.703e-04 | *loss* | 128 |
| | Flan T5 Large | TTC | 783,150,080 | 16 | 152 | 4 | 4 | 128 | 9.632e-05 | *loss* | 128 |
| | TinyStories 1M | TTC | 3,745,984 | 19 | 173 | 16 | 2 | 128 | 7.58e-04 | *loss* | 128 |
| | TinyStories 1M Instruct | TTC | 3,745,984 | 21 | 190 | 8 | 4 | 128 | 7.43e-04 | *loss* | 128 |
| | Pythia 14M | TTC | 14,067,712 | 18 | 113 | 16 | 4 | 128 | 5.89e-04 | *loss* | 128 |
| | GPT 2 | TTC | 124,439,808 | 25 | 92 | 32 | 1 | 128 | 8.215e-05 | *loss* | 128 |
| | GPT 2 Medium | TTC | 354,823,168 | 13 | 192 | 8 | 2 | 128 | 4.618e-05 | *loss* | 128 |
| | GPT 2 Large | TTC | 774,030,080 | 9 | 136 | 4 | 4 | 128 | 1.493e-05 | *loss* | 128 |
| | SmolLM2 135M | TTC | 134,515,008 | 7 | 104 | 8 | 2 | 128 | 1.08e-4 | *loss* | 64 |
| | SmolLM2 360M | TTC | 361,821,120 | 8 | 80 | 4 | 4 | 128 | 1.28e-04 | *loss* | 64 |
| | CodeGen | TTC | 356,712,448 | 6 | 172 | 8 | 2 | 128 | 6.798e-05 | *loss* | 64 |
| | Qwen3 | TTC | 596,049,920 | 7 | 94 | 8 | 2 | 128 | 2.804e-05 | *loss* | 96 |
| | DeepSeek Coder | TTC | 14,991,360 | 4 | 121 | 8 | 2 | 128 | 1.77e-04 | *loss* | 72 |
| | LLaMA 3.2 | TTC | 11,272,192 | 10 | 67 | 8 | 2 | 128 | 2.947e-05 | *loss* | 72 |
| | T5 Small | FIX | 60,506,624 | 24 | 100 | 16 | 2 | 128 | 1.5e-05 | *bleu* | 128 |
| *Luminaires* | T5 Small | TTC | 60,506,624 | 25 | 109 | 16 | 2 | 128 | 4.143e-04 | *loss* | 128 |
| | T5 Base | TTC | 222,903,552 | 20 | 137 | 4 | 4 | 128 | 1.863e-04 | *loss* | 128 |
| | T5 Large | TTC | 737,668,096 | 22 | 134 | 8 | 2 | 128 | 7.2e-05 | *loss* | 128 |
| | Flan T5 Small | TTC | 76,961,152 | 20 | 164 | 32 | 1 | 128 | 2.696e-04 | *loss* | 128 |
| | Flan T5 Base | TTC | 247,577,856 | 20 | 90 | 4 | 4 | 128 | 2.735e-04 | *loss* | 128 |
| | Flan T5 Large | TTC | 783,150,080 | 20 | 115 | 4 | 4 | 128 | 2.217e-04 | *loss* | 128 |
| | TinyStories 1M | TTC | 3,745,984 | 25 | 91 | 16 | 2 | 128 | 5.69e-04 | *loss* | 128 |
| | TinyStories 1M Instruct | TTC | 3,745,984 | 23 | 199 | 32 | 1 | 128 | 5.78e-04 | *loss* | 128 |
| | Pythia 14M | TTC | 14,067,712 | 25 | 132 | 32 | 2 | 128 | 7.52e-04 | *loss* | 128 |
| | GPT 2 | TTC | 124,439,808 | 20 | 70 | 8 | 2 | 128 | 9.35e-05 | *loss* | 128 |
| | GPT 2 Medium | TTC | 354,823,168 | 14 | 219 | 16 | 1 | 128 | 3.845e-05 | *loss* | 128 |
| | GPT 2 Large | TTC | 774,030,080 | 13 | 152 | 4 | 4 | 128 | 1.476e-05 | *loss* | 128 |
| | SmolLM2 135M | TTC | 134,515,008 | 8 | 123 | 8 | 2 | 128 | 1.352e-04 | *loss* | 64 |
| | SmolLM2 360M | TTC | 361,821,120 | 16 | 135 | 4 | 4 | 128 | 1.302e-04 | *loss* | 64 |
| | CodeGen | TTC | 356,712,448 | 9 | 135 | 16 | 1 | 128 | 5.268e-05 | *loss* | 64 |
| | Qwen3 | TTC | 596,049,920 | 9 | 70 | 4 | 4 | 128 | 2.985e-05 | *loss* | 96 |
| | DeepSeek Coder | TTC | 14,991,360 | 11 | 112 | 8 | 2 | 128 | 1.767e-04 | *loss* | 72 |
| | LLaMA 3.2 | TTC | 11,272,192 | 12 | 148 | 16 | 1 | 128 | 4.292e-05 | *loss* | 72 |
| | T5 Small | FIX | 60,506,624 | 23 | 100 | 16 | 2 | 128 | 1.5e-05 | *bleu* | 128 |

as their *as-is* robustness to accents and dialects is evaluated in this study. The *transformers* HuggingFace Python library[7] has been used for training, in addition to LORA[8] for DeepSeek and Llama variants. Data preparation for training includes tokenization and padding to a fixed 128 token length. Hyperparameter optimization has been performed using Optuna[9], so optimal values are chosen for ideal performance. Final hyperparameter used for training proposed models is shown in Table III.

### E. Evaluation

A total of 26 models have been evaluated in this study: 6 for transcribing (STT), 1 for translating the transcripts, 1 for fixing transcripts (FIX), and 18 for transforming instructions into commands (TTC). Used evaluation metrics are shown in Table IV.

Transcript quality, before and after being fixed by the FIX component, is evaluated employing WER, Levenshtein distance (LEV) (word-level), and semantic textual similarity (STS) metrics, as most of the original meaning of speech data should remain in the obtained transcript and be fed to the TTC component. Command rightfulness, obtained from the TTC component, is evaluated employing Exact match (EQ) and LEV (character-level) metrics. Command generation is evaluated in a strict manner, and high performance in the mentioned metrics is expected.

Results on the mentioned metrics, exposed in Sections IV-A and IV-B, would highlight the differences between the proposed models and enable the selection of those to compose the final system described in Section III-A. In addition, in Section IV-C a functional implementation of

TABLE IV
TEXT SIMILARITY METRICS USED ON EVALUATION

| Metric | Description | Ref. |
|---|---|---|
| WER | Word Error Rate. Proportion of word errors to words processed. | [25] |
| LEV | Levenshtein Distance. Number of insertions, deletions and substitutions required to transform a sentence into another. | [53] |
| STS | Semantic Textual Similarity. Cosine distance between pairs of sentence embeddings calculated with 'sentence-transformer' library. | [54] |
| EQ | 'Equals' or 'Exact Match'. Whether a sentence is completely equal to another. | [55] |

[7]https://huggingface.co
[8]https://github.com/microsoft/LoRA
[9]https://optuna.org/

TABLE V
EVALUATION OF COMMAND GENERATION, BEFORE AND AFTER TRAINING PROPOSED MODELS

| Models | *Docker* | | | | *Luminaires* | | | | Time[1](s) |
|---|---|---|---|---|---|---|---|---|---|
| | EQ (%) | | LEV (%) | | EQ (%) | | LEV (%) | | |
| | *Before* | *After* | *Before* | *After* | *Before* | *After* | *Before* | *After* | |
| T5 Small | 0 | 76.69 | 38.40 | 95.72 | 0 | 89.62 | 27.12 | 98.22 | 0.20 |
| T5 Base | 0 | 89.85 | 40.48 | 98.43 | 0 | 94.34 | 30.86 | 99.68 | 0.43 |
| T5 Large | 0 | **92.86** | 17.07 | **99.10** | 0 | **97.17** | 9.30 | **99.86** | 1.82 |
| Flan T5 Small | 0 | 75.56 | 38.31 | 95.17 | 0 | 85.85 | 29.61 | 98.57 | 0.35 |
| Flan T5 Base | 0 | 90.23 | 32.79 | 98.65 | 0 | 92.45 | 27.60 | 99.06 | 0.54 |
| Flan T5 Large | 0 | 92.48 | 33.44 | 98.92 | 0 | 94.34 | 27.97 | 99.62 | 1.38 |
| TinyStories 1M | 0 | 41.72 | 0 | 85.26 | 0 | 30.19 | 0 | 82.36 | **0.10** |
| TinyStories 1M Instruct | 0 | 50.38 | 0 | 89.20 | 0 | 33.96 | 0 | 83.96 | 0.12 |
| Pythia 14M | 0 | 40.23 | 0.85 | 86.03 | 0 | 41.51 | 0.34 | 85.60 | 0.13 |
| GPT2 | 0 | 6.77 | 19.42 | 41.62 | 0 | 13.21 | 10.41 | 62.11 | 1.37 |
| GPT2 Medium | 0 | 53.01 | 19.11 | 86.88 | 0 | 27.36 | 11.16 | 65.95 | 3.62 |
| GPT2 Large | 0 | 65.04 | 20.13 | 89.32 | 0 | 8.49 | 11.30 | 45.06 | 7.12 |
| SmolLM2 135M | 0 | 83.08 | 28.70 | 96.59 | 0 | 89.62 | 16.51 | 97.02 | 0.60 |
| SmolLM2 360M | 0 | 89.47 | 17.07 | 98.20 | 0 | 95.28 | 10.89 | 98.10 | 0.90 |
| CodeGen Mono | 0 | 84.96 | 3.32 | 97.26 | 0 | 89.62 | 3.91 | 98.37 | 3.13 |
| Qwen3[2] | 0 | 43.23 | 26.63 | 87.53 | 0 | 33.02 | 15.51 | 60.39 | $\infty$ |
| DeepSeek Coder Instruct[2] | 0 | 85.34 | 28.59 | 97.38 | 0 | 85.85 | 13.59 | 99.09 | $\infty$ |
| Llama 3.2 Instruct[2] | 0 | 27.44 | 31.53 | 81.92 | 0 | 7.55 | 17.38 | 67.36 | $\infty$ |

[1] Time to process a single input sentence of 40 characters using a conventional laptop (Processor 11th Gen Intel Core i5-1135G7, 16GiB memory, Ubuntu 24.04 LTS, no CUDA support). Models which cannot be run by this device are annotated with $\infty$ symbol.
[2] Due to their high computational cost, these models are not practical alternatives for running on the Edge, being shown only for accuracy comparison.

this study voice assistant is reported, along with usage of target devices' resources (CPU, RAM, and GPU) and models inference time, since the proposed system should run locally on an edge device as previously stated.

## IV. RESULTS

### A. TTC Component

Several models have been evaluated for composing the TTC component of this study proposed system for a voice assistant. Trained models are to demonstrate their improvement in performance for command generation by the fine-tuning process, while also providing comparative benchmark metrics that would guide the selection of the most appropriate model according to hardware constraints in edge environments. Evaluation results are shown in Table V. EQ results show the generated commands that could be directly executed on a terminal, while LEV results report how similar predicted commands are to their ground truths.

All models benefit from being trained on the custom datasets exposed in this study, as none of them are able to predict the rightful command given a proper instruction before training. Even more, their generated commands are only ~18% similar at most to what they should be. Regarding Docker command context, T5 Large achieves the best results with 92.86% exact matches and 99.10% text similarity. Overall, *text to text* models obtain 86.28% mean exact matches while *text generation* models only achieve 55.89%. These results demonstrate that *text to text* models sequential input–output formula aligns perfectly with the task of transforming natural language instructions into commands. Similar results are achieved on the luminaires command context, in which *text to text* and *text generation* models obtain 92.29% and 46.31% exact matches, respectively, with T5 variants as the best performers. Among the *text to text* models proposed in this study, T5 variants seem to fit slightly better to the TTC task in comparison with its derived model, Flan T5, with 90.09% and 88.48% mean exact matches, respectively. In general, the T5 family demonstrates outstanding performance for this task, not only with its larger variants but also the more compact ones, surpassing 75% of exact matches with at least half of the parameters of *text generation* models, highlighting its suitability for resource-constrained environments without compromising precision. While they are not the quickest, T5 variants also achieve suitable inference times on CPU-only devices, with T5 Small and T5 Base obtaining 0.20 and 0.43 s, respectively. T5 Large, considering its greater size, achieves 1.82 s of inference time, yet acceptable for real-time applications. In contrast, larger models could not be executed on CPU-only edge hardware under the tested conditions, as their computational and memory requirements exceed the capabilities of such devices without GPU acceleration. Although the comparison between *text to text* and *text generation* models has already been addressed, this study results demonstrate that employing larger SLMs or even LLMs for the TTC component does not guarantee better results in comparison with smaller approaches, demonstrating that the number of learnable parameters and/or FLOPS have an impact on performance across the same model architecture, but not across different ones. Including them in the comparative aims to highlight the differences in performance for command generation and to ensure that smaller alternatives are suitable to run on the edge and still obtain high accuracy.

The evaluation of the TTC component highlights the impact of model size, computational cost, and command generation accuracy for designing SLMs for edge applications under realistic hardware constraints. Overall, these benchmark results confirm that fine-tuned SLMs provide a robust and efficient solution, particularly the T5 variants. Building on this analysis, Section III-B examines the integration of the STT and FIX components into the proposed system, which introduces addi-

TABLE VI
EVALUATION OF TRANSCRIPTS QUALITY AND PERFORMANCE

| STT Model | Dataset | Lang. | Task | N. Samples | Prompt WER | Prompt LEV (%) | Prompt STS (%) | Command EQ (%) | Time[1](s) |
|---|---|---|---|---|---|---|---|---|---|
| Whisper Tiny | *Docker* | EN | Transcribe | 3190[2] | 28.66 | 76.06 | 80.05 | 42.38 | 0.87 |
| | *Luminaires* | EN | Transcribe | 530 | 42.70 | 63.81 | 80.93 | 56.41 | |
| | | ES | Transcribe | 530 | 29.59 | 74.18 | 84.32 | *null*[4] | |
| | | ES | Translate | 514[2] | *null*[3] | *null*[3] | 54.90 | 20.82 | **0.76** |
| Whisper Base | *Docker* | EN | Transcribe | 3192 | 23.81 | 79.79 | 83.22 | 48.09 | 1.42 |
| | *Luminaires* | EN | Transcribe | 530 | 39.58 | 65.30 | 83.28 | 62.26 | |
| | | ES | Transcribe | 530 | 29.69 | 73.79 | 83.81 | *null*[4] | |
| | | ES | Translate | 519[2] | *null*[3] | *null*[3] | 65.15 | 44.32 | 1.40 |
| Whisper Small | *Docker* | EN | Transcribe | 3192 | 17.03 | 85.15 | 89.29 | 58.52 | 4.42 |
| | *Luminaires* | EN | Transcribe | 530 | 32.02 | 71.20 | 88.17 | 74.53 | |
| | | ES | Transcribe | 530 | 20.08 | 82.72 | 90.29 | *null*[4] | |
| | | ES | Translate | 530 | *null*[3] | *null*[3] | 74.07 | 67.17 | 3.98 |
| Whisper Medium | *Docker* | EN | Transcribe | 3192 | 14.67 | 86.96 | 90.86 | **64.35** | 14.32 |
| | *Luminaires* | EN | Transcribe | 530 | 29.11 | 73.32 | 90.73 | **78.87** | |
| | | ES | Transcribe | 530 | 17.24 | 85.19 | 92.18 | *null*[4] | |
| | | ES | Translate | 530 | *null*[3] | *null*[3] | 80.65 | 73.77 | 11.26 |
| Wav2Vec Base | *Docker* | EN | Transcribe | 3192 | 55.68 | 50.56 | 50.81 | 8.84 | 0.50 |
| | *Luminaires* | EN | Transcribe | 530 | 55.24 | 50.85 | 59.83 | 23.96 | |
| | | ES | Transcribe | 530 | 22.49 | 79.50 | 79.69 | *null*[4] | |
| | | ES | Translate[5] | 523[2] | *null*[3] | *null*[3] | 65.82 | 41.49 | 1.38 |
| Conformer Large | *Docker* | EN | Transcribe | 3192 | 29.33 | 73.84 | 75.29 | 26.94 | 0.49 |
| | *Luminaires* | EN | Transcribe | 530 | 30.38 | 73.52 | 78.79 | 51.70 | |
| | | ES | Transcribe | 530 | 10.12 | 91.11 | 91.87 | *null*[4] | |
| | | ES | Translate[5] | 522[2] | *null*[3] | *null*[3] | 80.19 | 62.26 | 1.51 |

[1] Time to process (transcribe or translate) an audio file of 2.53s using a conventional laptop (Processor 11th Gen Intel Core i5-1135G7, 16GiB memory, Ubuntu 24.04 LTS, no CUDA support).
[2] Samples contained hallucinations, which have been automatically removed to obtain metric results.
[3] Translation cannot be evaluated as simple text similarity, but exclusively semantic similarity.
[4] TTC component is trained on English data.
[5] Translation made by OPUS MT ES-EN model.

tional variability in performance and cost, further illustrating the balance required to deploy voice assistants on the edge.

### B. STT and FIX Components

Different STT models have been tested to transcribe and translate audio data containing instructions describing Docker and emergency luminaires system commands (see Section III-C). Evaluation results are shown in Table VI. WER, LEV, and STS result report how similar (or not) are generated transcripts are to their ground truths, with STS being the only metric considering semantic likeness. Obtained transcripts in English have been tested on T5 Large trained for TTC (see Section IV-A), which produce the EQ results.

No relevant impact of recorded subjects' specific age group, gender, language accent, or dialect has been noticed on the results. Larger Whisper variants seem to perform better in all prompt similarity metrics. Considering both evaluated datasets transcripts from English audio data, Whisper Medium, as the best performer, achieved 16.73 WER, 85.02% LEV, and 90.85% STS. It is followed by Whisper Small (19.17 WER, 83.13% LEV, and 89.14% STS), Whisper Base (26.06 WER, 77.73% LEV, and 83.23% STS), Whisper Tiny (30.66 WER, 74.32% LEV, and 80.17% STS), Conformer Large (29.48 WER, 73.79% LEV, and 75.78% STS), and Wav2Vec2 Base (55.62 WER, 50.60% LEV, and 52.10% STS). This trend is also observed in command exact match results from the TTC component, as Whisper Medium obtains 61.96% weighted mean of exact matches, also followed by Whisper Small (56.68% EQ), Whisper Base (47.02% EQ), Whisper Tiny (41.64% EQ), Conformer Large (30.47% EQ), and Wav2Vec2 Base (10.99% EQ). However, if English translations from Spanish audio data are taken into account, Whisper Medium and Whisper Small weighted mean EQ results increase 1.45% and 1.30%, respectively, while Whisper Base and Whisper Tiny performance decreases 0.31% and 2.50%, respectively. This finding shows that larger Whisper variants translate better to English text than smaller ones. Conformer Large and Wav2Vec2 weighted mean EQ results on English texts increase ~3.8% when taking into account Spanish audio data, whose transcripts are translated by using the OPUS MT model. Regarding STT task comparison, better results are generally obtained when transcribing English audio data (80.29% STS and 57.58% EQ), even from nonnative speakers, than translating Spanish audio data into English text (70.21% STS and 51.88% EQ). In terms of language, better results are achieved when transcribing Spanish audio data from native speakers (21.53 WER, 81.08% LEV, and 87.03% STS) than English audio data from nonnative speakers (38.17 WER, 66.33% LEV, and 80.29% STS). Wav2Vec2 Base achieves only 56.66% STS and 13.68% weighted mean of exact command matches, being the worst performer in all scopes. Conformer Large, in comparison with Whisper Base, as both have comparable number of parameters, achieves poor results when transcribing English audio data (−7.45% STS and −16.55% EQ), while obtaining better transcripts from Spanish audio data (+ 8.06% STS) although not correct enough to obtain better exact matches results (−11.89% EQ).

TABLE VII
EVALUATION OF ENGLISH TRANSCRIPTS QUALITY, BOTH TRANSCRIBED AND TRANSLATED, BY USING THE FIX COMPONENT

| Dataset | STT Model | N. Samples[1] | Output | Prompt WER[2] | Prompt LEV[2](%) | Prompt STS (%) | Command EQ (%) |
|---|---|---|---|---|---|---|---|
| *Docker* | Whisper Tiny | 1199 | Raw | 16.37 | 86.48 | 89.04 | 60.13 |
| | | | Fixed | 1.39 | 99.04 | 99.27 | 81.23 |
| | Whisper Base | 1295 | Raw | 11.68 | 90.17 | 91.63 | 62.78 |
| | | | Fixed | 1.49 | 99.09 | 99.24 | 79.92 |
| | Whisper Small | 1574 | Raw | 7.00 | 94.01 | 95.85 | 71.03 |
| | | | Fixed | 0.94 | 99.43 | 99.64 | 81.89 |
| | Whisper Medium | 1705 | Raw | 5.14 | 95.42 | 96.89 | 74.43 |
| | | | Fixed | 0.73 | 99.54 | 99.75 | **83.22** |
| | Wav2Vec2 Base | 672 | Raw | 52.54 | 53.28 | 53.22 | 14.14 |
| | | | Fixed | 3.61 | 97.13 | 97.54 | 75.89 |
| | Conformer Large | 948 | Raw | 19.40 | 82.55 | 83.72 | 44.83 |
| | | | Fixed | 1.73 | 98.87 | 99.21 | 80.80 |
| *Luminaires* | Whisper Tiny | 228 | Raw | 65.01 | 52.14 | 71.27 | 45.61 |
| | | | Fixed | 12.39 | 91.13 | 94.92 | 82.02 |
| | Whisper Base | 238 | Raw | 56.73 | 56.26 | 76.91 | 58.40 |
| | | | Fixed | 3.68 | 97.07 | 98.21 | 92.86 |
| | Whisper Small | 248 | Raw | 47.35 | 63.49 | 84.29 | 76.21 |
| | | | Fixed | 3.08 | 97.82 | 98.95 | 91.94 |
| | Whisper Medium | 277 | Raw | 41.21 | 67.09 | 88.14 | 80.51 |
| | | | Fixed | 2.11 | 98.48 | 99.38 | **94.22** |
| | Wav2Vec2 Base | 215 | Raw | 57.73 | 53.22 | 64.63 | 34.88 |
| | | | Fixed | 7.02 | 94.47 | 95.28 | 84.19 |
| | Conformer Large | 242 | Raw | 40.82 | 69.90 | 82.41 | 63.64 |
| | | | Fixed | 2.00 | 98.55 | 99.24 | 91.74 |

[1] Samples from *train* split of FIX datasets have been removed.
[2] Only transcribe task.

Both Wav2Vec2 Base and Conformer Large, along with the OPUS MT model, achieve better results when translating Spanish audio data if compared to Whisper Base (+ 0.67% and +15.04% STS, respectively), yielding that combining distinct STT and translation models seems to outperform all-in-one architectures like Whisper. However, using the OPUS MT model for translation increases ∼1 s the overall processing time, while all Whisper models obtain similar inference times despite performing different tasks, even taking less time to translate audio data to English texts.

Despite yielding state-of-the-art results with lower values of WER metrics in their respective original research, the proposed models perform poorly to obtain a rightful transcript in order to use it as input for the TTC component. As already mentioned in this study, a component for fixing STT transcripts is proposed to be included in the final system, which would only add 0.20 s to the overall inference time. Therefore, T5 Small has been trained on the datasets described in Section III-C3 and compared to original metrics results, as shown in Table VII. All proposed models improve their transcripts quality across all evaluated datasets after the FIX component is used: Whisper Medium (−9.26 WER, +7.93% LEV, +4.03% STS, and +9.53% EQ), Whisper Small (−11.26 WER, +9.35% LEV, +5.28% STS, and +11.53% EQ), Whisper Base (−16.84 WER, +13.87% LEV, +9.74% STS, and +19.83% EQ), Whisper Tiny (−20.99 WER, +16.78% LEV, +12.37% STS, and +23.55% EQ), Wav2Vec2 Base (−49.36 WER, +43.22% LEV, +41.15% STS, and +58.73% EQ), and Conformer Large (−21.97 WER, +18.83% LEV, +15.76% STS, and +34.37% EQ). Smaller variants report the greatest improvements, as they demonstrated poor results before the usage of the FIX component. Considering only the number of exact matches of generated commands by the TTC component, Wav2Vec2 Base fixed transcripts are better than Whisper Medium original transcripts, highlighting the importance of the FIX component within this study system. Beyond variants comparison, fixed transcripts quality exceeds original WER (2.23 < 23.84), LEV (98.41% > 80.08%), STS (98.88% > 84.15%), and EQ (82.05% > 55.79%) results.

These findings show a progressive improvement in performance as model size increases, but also differences across model architectures. Whisper variants overall outperform Wav2Vec2 and Conformer, offering acceptable inference times and better multilingual robustness under nonnative speakers and nonstandard accents. Significant improvements are obtained by employing the FIX component for fixing transcript errors (+ 30% exact command matches) while just increasing by 0.2 s the overall inference time.

### C. Use Case

In Sections IV-A and IV-B a comprehensive evaluation and comparison between proposed models have been shown in order to be able to select the required ones for composing TTC, STT, and FIX components, so this study system (see Section III-A) could be built subject to hardware requirements of the target edge environment. Particularly, this study proposes a use case in which an IoT developer or technician team can use this system to perform deployment and maintenance operations comfortably from their own laptop. Two different devices are proposed for testing: PC1 is a more powerful desktop device, ideally used by the IoT developer or technician team, and PC2 is a constrained portable PC with no graphical card compatible with CUDA, more likely to be used by the maintenance team for checking on-site the IoT deployment status. T5 Large has been selected to conform to the TTC

TABLE VIII
RESOURCE USAGE BY PROPOSED SYSTEM

| Device | STT Model | Time (s) | CPU (%) | | MEM (%) | | GPU (%) | | GPU MEM (%) | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | *mean* | *median* | *mean* | *median* | *mean* | *median* | *mean* | *median* |
| *PC1*[1] | *Baseline* | | 0.09 | 0.10 | 19.44 | 19.40 | 0.00 | 0.0 | 0.03 | 0.03 |
| | Whisper Tiny | 0.63 | 43.06 | 53.20 | 23.61 | 24.00 | 7.86 | 1.0 | 0.06 | 0.06 |
| | Whisper Base | 0.52 | 44.90 | 51.65 | 23.80 | 23.80 | 9.50 | 0.0 | 0.08 | 0.08 |
| | Whisper Small | 0.61 | 39.63 | 43.50 | 24.29 | 24.30 | 19.29 | 18.0 | 0.15 | 0.15 |
| | Whisper Medium | 0.85 | 29.77 | 31.60 | 24.40 | 24.40 | 51.11 | 70.0 | 0.36 | 0.36 |
| | Wav2Vec2 Base | 0.56 | 45.10 | 53.10 | 26.20 | 26.30 | 1.83 | 0.0 | 0.34 | 0.34 |
| | Conformer Large | 0.53 | 45.55 | 52.60 | 30.73 | 30.70 | 6.67 | 4.5 | 0.14 | 0.14 |
| *PC2*[2] | *Baseline* | | 1.68 | 0.80 | 48.08 | 48.10 | *null*[3] | *null*[3] | *null*[3] | *null*[3] |
| | Whisper Tiny | 2.80 | 53.54 | 53.15 | 49.05 | 49.00 | *null*[3] | *null*[3] | *null*[3] | *null*[3] |
| | Whisper Base | 3.22 | 52.08 | 51.80 | 49.12 | 49.10 | *null*[3] | *null*[3] | *null*[3] | *null*[3] |
| | Whisper Small | 6.04 | 53.24 | 52.40 | 49.46 | 49.30 | *null*[3] | *null*[3] | *null*[3] | *null*[3] |
| | Whisper Medium | 15.55 | 53.71 | 51.80 | 47.84 | 48.30 | *null*[3] | *null*[3] | *null*[3] | *null*[3] |
| | Wav2Vec2 Base | 3.83 | 56.98 | 58.40 | 48.25 | 48.30 | *null*[3] | *null*[3] | *null*[3] | *null*[3] |
| | Conformer Large | 3.15 | 58.98 | 63.80 | 52.05 | 52.05 | *null*[3] | *null*[3] | *null*[3] | *null*[3] |

[1] Processor 12th Gen Intel Core i7-12700F, 16GiB memory, NVIDIA GeForce RTX 3060, Ubuntu 22.04 LTS.
[2] Processor 11th Gen Intel Core i5-1135G7, 16GiB memory, no CUDA support, Ubuntu 24.04 LTS.
[3] No GPU has been used on *PC2* inference.

component for this use case as it achieves 95.02% of mean exact matched, requiring only 1.82 s when run on CPU. For the STT component, all proposed STT models are to be tested, as all of them improve their performance if used along with T5 Small for the FIX component, while only increasing 0.20 s the inference time when run on CPU. Therefore, PC1 graphical card has been employed to run the STT component, while the rest of the components have to be run on the CPU processor. In the case of PC2, all components must run on the CPU processor.

Inference time per audio file processed and average system resources usage are shown in Table VIII. PC1 spends ~0.62 s to process a given instruction, regarding the model used for the STT component. PC2, as the most constrained device, takes from 2.80 to 15.55 s to process a single prompt. Regarding resource usage, PC1 offers a more balanced distribution of workload. By using one GPU for STT inference, it reduces CPU and RAM usage by a ~11% and a ~25%, respectively. However, using lighter alternatives than Whisper Medium or Whisper Small for the STT component and loading them on a GPU does not result in as much improvement; in fact, CPU workload is similar compared to PC2 performance in these cases. This may be caused either by the inefficient use of the GPU resulting from using such small models, which leads to additional synchronization processes, or by the increased demand, as fast inferences are performed, placed on the next components in the pipeline, FIX and TTC, which are loaded on the CPU.

In conclusion, the proposed system in this study requires reasonable device resources and consumes a decent average inference time for this use case, being a local privacy-preserving solution for a voice assistant for IoT deployment, completely run on edge devices. The voice assistant would process a given instruction with ~85% accuracy in a maximum of ~15 s if using Whisper Medium, the best performer as the STT component, although smaller alternatives are also suitable, requiring less inference time (−13 s) and still obtaining great results (up to ~81% exact command matches). This solution guarantees a stable usage of device memory while having a medium impact on CPU (+ 45%), being improvable if any GPU is available.

## V. CONCLUSION

This study focuses on the following research question: *How effective are fine-tuned SLMs combined with embedded STT models in generating accurate commands for IoT infrastructure maintenance by nonnative English speakers in offline, resource-constrained environments?* In order to address it, this study evaluates different state-of-the-art STT, *text to text*, and *text generation* models with the objective of designing a cost-effective privacy-preserving voice assistant locally run on edge devices. The assistant would aid IoT solutions technicians in usual deployment or maintenance operations. The proposed system for the voice assistant, as outlined in Fig. 1, include three components: STT, for transcribing and translating into English given audio data into text; FIX (*spell checker*), for correcting spell errors found in transcripts obtained from the STT component; and TTC, to transform a given textual instruction into a command ready to be executed. In order to build them, several datasets have been manually drafted, as described in Section III-C, for training or evaluating the proposed models. In Sections IV-A and IV-B, a benchmark and its results for comparing the proposed models with different standard and well-known metrics are exposed. A functional implementation of this study's proposed use case has been addressed in Section IV-C by employing models from the previous evaluation process. This voice assistant for command generation would obtain a rightful command from a given instruction in 2.80 s, without needing to use any GPU, with ~81% accuracy. If any GPU is available in the target edge environment, a more powerful STT model could be employed, and this system's accuracy would increase up to ~85%, obtaining the generated command in only 0.85 s. This solution offers a reasonable inference time and resource usage for a local privacy-preserving solution.

This study's findings show that carefully fine-tuned SLMs can deliver high efficiency for speech-driven command gen-

eration in IoT deployment maintenance. However, further challenges and future work must be addressed. Language and accent barriers yet exist for STT models, being more precise at processing native speakers' voice data, even more so if they speak standard accents. Broader multilingual datasets that gather spontaneous and noisy audio data from nonstandard accents are required to fine-tune STT models so their robustness is increased. This study evaluated *text to text* and *text generation* models to generate commands from a given instruction, although only two command contexts have been tested. The proposed voice assistant should offer aid with other IoT tools and environments, such as Kubernetes or LoRaWAN, thus requiring datasets comprising samples from those. Moreover, building the complete proposed agent-based system will enable providing deployment context to given instructions, using retrieval-augmented generation (RAG) techniques to search for command information, and, in general, a conversational assistant capable of reasoning and handling long and even ambiguous instructions. Finally, user acceptance and usability studies must be conducted on the final system, although users are expected to confirm each generated command.

## AUTHORSHIP CONTRIBUTION STATEMENT

Ángela Gijón Flores: Investigation and writing–original draft. Cristina Bolaños Peño: Investigation, data curation, writing-review, and editing. Henry Llumiguano Solano: Data curation and methodology. Jesús Fernández-Bermejo Ruiz: Writing–review and editing. Félix Jesús Villanueva Molina: Supervision and funding acquisition. Fernando Rincón Calle: Conceptualization and resources.

## ACKNOWLEDGMENT

Participants of the voice recording from the ARCO Research Group (University of Castilla–La Mancha, Ciudad Real, Spain) are acknowledged for their contributions and support while conducting this study.

## REFERENCES

[1] R. B. Cabot, "Re-imagining the command line user experience for problem solving," Dept. Comput. Sci., Univ. Bath, Bath, U.K., Tech. Rep., May 2017.

[2] D. Guo et al., "Deepseek-coder: When the large language model meets programming—The rise of code intelligence," 2024, *arXiv:2401.14196*.

[3] A. Grattafiori et al., "The llama 3 herd of models," 2024, *arXiv:2407.21783*.

[4] C. Cao, F. Wang, L. Lindley, and Z. Wang, "Managing Linux servers with LLM-based AI agents: An empirical evaluation with GPT4," *Mach. Learn. Appl.*, vol. 17, Sep. 2024, Art. no. 100570.

[5] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly," *High-Confidence Comput.*, vol. 4, no. 2, Jun. 2024, Art. no. 100211.

[6] L. Chen and G. Varoquaux, "What is the role of small models in the LLM era: A survey," 2024, *arXiv:2409.06857*.

[7] J. R. Bellegarda, "Spoken language understanding for natural interaction: The siri experience," in *Natural Interaction With Robots, Knowbots and Smartphones*, J. Mariani, S. Rosset, M. Garnier-Rizet, and L. Devillers, Eds., New York, NY, USA: Springer, 2014, pp. 3–14.

[8] A. Radford, J. Wook Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," 2022, *arXiv:2212.04356*.

[9] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," 2020, *arXiv:2006.11477*.

[10] C. Graham and N. Roll, "Evaluating OpenAI's whisper ASR: Performance analysis across diverse accents and speaker traits," *JASA Exp. Lett.*, vol. 4, no. 2, Feb. 2024, Art. no. 025206.

[11] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," 2022, *arXiv:2210.03629*.

[12] P. Belcak et al., "Small language models are the future of agentic AI," 2025, *arXiv:2506.02153*.

[13] M. S. Abdul, S. M. Sam, N. Mohamed, and R. A. Dziyauddin, "Docker containers usage in the Internet of Things: A survey," *Open Int. J. Informat.*, vol. 7, no. 2, pp. 208–220, 2019.

[14] A. Al-Zuraiqi and D. Greer, "Evaluating alignment techniques for enhancing LLM performance in a closed-domain application: A RAG bench-marking study," in *Proc. Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2024, pp. 1170–1175.

[15] A. Gijón, C. Bolaños, and F. Villanueva, "From voice to shell: Datasets," Zenodo, Jul. 2025, doi: 10.5281/zenodo.17288923.

[16] R. Bommasani et al., "On the opportunities and risks of foundation models," Stanford Center Res. Found. Models, Stanford, CA, USA, Tech. Rep. CS-FM-01, Aug. 2021.

[17] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2022, pp. 5998–6008.

[18] R. Wang, Z. Gao, L. Zhang, S. Yue, and Z. Gao, "Empowering large language models to edge intelligence: A survey of edge efficient LLMs and techniques," *Comput. Sci. Rev.*, vol. 57, Aug. 2025, Art. no. 100755.

[19] Z. Lu et al., "Small language models: Survey, measurements, and insights," 2024, *arXiv:2409.15790*.

[20] D. Hillier, L. Guertler, C. Tan, P. Agrawal, C. Ruirui, and B. Cheng, "Super tiny language models," 2024, *arXiv:2405.14159*.

[21] R. Eldan and Y. Li, "TinyStories: How small can language models be and still speak coherent english?," 2023, *arXiv:2305.07759*.

[22] Y. Gong, G. Liu, Y. Xue, R. Li, and L. Meng, "A survey on dataset quality in machine learning," *Inf. Softw. Technol.*, vol. 162, Oct. 2023, Art. no. 107268. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584923001222

[23] C. Hao, J. Dotzel, J. Xiong, L. Benini, Z. Zhang, and D. Chen, "Enabling design methodologies and future trends for edge AI: Specialization and codesign," *IEEE Design Test*, vol. 38, no. 4, pp. 7–26, Aug. 2021.

[24] E. Rubio-Drosdov, D. Díaz-Sánchez, F. Almenárez, P. Arias-Cabarcos, and A. Marín, "Seamless human-device interaction in the Internet of Things," *IEEE Trans. Consum. Electron.*, vol. 63, no. 4, pp. 490–498, Nov. 2017.

[25] A. C. Morris, V. Maier, and P. Green, "From WER and RIL to MER and WIL: Improved evaluation measures for connected speech recognition," in *Proc. Interspeech*, Oct. 2004, pp. 2765–2768.

[26] M. Post, "A call for clarity in reporting BLEU scores," in *Proc. 3rd Conf. Mach. Translation: Res. Papers*, Belgium, Oct. 2018, pp. 186–191.

[27] A. Gulati et al., "Conformer: Convolution-augmented transformer for speech recognition," in *Proc. Interspeech*, Oct. 2020, pp. 5036–5040.

[28] T. Palmeira Ferraz, M. Zanon Boito, C. Brun, and V. Nikoulina, "Multilingual DistilWhisper: Efficient distillation of multi-task speech models via language-specific experts," 2023, *arXiv:2311.01070*.

[29] A. Koenecke, A. S. G. Choi, K. X. Mei, H. Schellmann, and M. Sloane, "Careless whisper: Speech-to-text hallucination harms," in *Proc. ACM Conf. Fairness, Accountability, Transparency*, Jun. 2024, pp. 1672–1681.

[30] M. Giudici, A. Sironi, I. Villa, S. Scherini, and F. Garzotto, "Generating HomeAssistant automations using an LLM-based chatbot," 2025, *arXiv:2505.02802*.

[31] S. Gallo, F. Paternó, and A. Malizia, "A conversational agent for creating automations exploiting large language models," *Pers. Ubiquitous Comput.*, vol. 28, no. 6, pp. 931–946, Dec. 2024.

[32] F. Westenfelder, E. Hemberg, S. Moskal, U.-M. O'Reilly, and S. Chiricescu, "LLM-supported natural language to bash translation," in *Proc. Conf. Nations Americas Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, 2025, pp. 11135–11147.

[33] B. Xiao, B. Kantarci, J. Kang, D. Niyato, and M. Guizani, "Efficient prompting for LLM-based generative Internet of Things," *IEEE Internet Things J.*, vol. 12, no. 1, 2024.

[34] H. Cui, Y. Du, Q. Yang, Y. Shao, and S. C. Liew, "LLMind: Orchestrating AI and IoT with LLM for complex task execution," *IEEE Commun. Mag.*, vol. 63, no. 4, pp. 214–220, Apr. 2025.

[35] S. Bharadwaj and S. Shevade, "Explainable natural language to bash translation using abstract syntax tree," in *Proc. 25th Conf. Comput. Natural Lang. Learn.*, 2021, pp. 258–267.

[36] M. Agarwal et al., "CLAI: A platform for AI skills on the command line," 2020, *arXiv:2002.00762*.
[37] S. Bharadwaj and S. Shevade, "Efficient constituency tree based encoding for natural language to bash translation," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2022, pp. 3159–3168.
[38] M. Asif, T. Ahmed Khan, and W.-C. Song, "Leveraging cognitive machine reasoning and NLP for automated intent-based networking and e2e service orchestration," *IEEE Access*, vol. 13, pp. 19456–19468, 2025.
[39] K. Balázs Neszlényi, A. Milos, and A. Kiss, "AssistantGPT: Enhancing user interaction with LLM integration," in *Proc. IEEE 22nd Jubilee Int. Symp. Intell. Syst. Informat. (SISY)*, Sep. 2024, pp. 000619–000624.
[40] C. Qian et al., "Autogen: Enabling next-generation llm applications via multi-agent conversation framework," 2023, *arXiv:2308.08155*.
[41] S. Liu, H. Shen, S. Che, M. Ghandi, and M. Li, "HERA: Hybrid edge–cloud resource allocation for cost-efficient AI agents," 2025, *arXiv:2504.00434*.
[42] I. Ong et al., "RouteLLM: Learning to route LLMs with preference data," 2024, *arXiv:2406.18665*.
[43] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," 2025, *arXiv:2503.18666*.
[44] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2019.
[45] H. Won Chung et al., "Scaling instruction-finetuned language models," 2022, *arXiv:2210.11416*.
[46] S. Biderman et al., "Pythia: A suite for analyzing large language models across training and scaling," 2023, *arXiv:2304.01373*.
[47] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI Blog, Feb. 2019, vol. 1, no. 8.
[48] L. B. Allal et al., "SmolLM2: When SMOL goes big–data-centric training of a small language model," 2025, *arXiv:2502.02737*.
[49] E. Nijkamp et al., "CodeGen: An open large language model for code with multi-turn program synthesis," 2022, *arXiv:2203.13474*.
[50] A. Yang et al., "Qwen3 technical report," 2025, *arXiv:2505.09388*.
[51] J. Tiedemann and S. Thottingal, "OPUS-MT–building open translation services for the world," in *Proc. 22nd Annu. Conf. Eur. Assoc. Mach. Transl. (EAMT)*, Lisbon, Portugal, 2020, pp. 479–480.
[52] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," 2023, *arXiv:2311.05232v*.
[53] RapidFuzz. *Levensthein Python Library 0.27.1*. Accessed: Jun. 30, 2025. [Online]. Available: https://github.com/rapidfuzz/Levenshtein/tree/v0.27.1
[54] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3980–3990.
[55] IBM. *Exact Match Exaluation Metric*. Accessed: Jun. 30, 2025. [Online]. Available: https://www.ibm.com/docs/en/watsonx/saas?topic=metrics-exact-match



**Ángela Gijón Flores** received the B.S. degree in computer engineering from the University of Castilla–La Mancha (UCLM), Ciudad Real, Spain, in 2025.

She is currently a Researcher at the ARCO Group, UCLM. Her current research interests include the Internet of Things (IoT) and natural language processing (NLP), particularly focused on small language models (SLMs).



**Cristina Bolaños Peño** received the B.S. degree in computer engineering and the M.Sc. degree in computer engineering from the University of Castilla–La Mancha (UCLM), Ciudad Real, Spain, in 2019 and 2021, respectively, where she is currently pursuing the Ph.D. degree in advanced computing technologies.

Her current research interests include the Internet of Things (IoT), embedded systems, and natural language processing (NLP).

Ms. Bolaños Peño was a recipient of the "SCIE-ZONTA-SNGULAR" Award from Spanish Scientific IT Society (SCIE) in 2023, which acknowledges the contribution of women in the field of computer science.



**Henry Llumiguano Solano** received the B.Sc. degree in mechatronic engineering from the Equinox University of Technology, Quito, Ecuador, in 2018, and the M.Sc. degree in distributed renewable energies from the University of Córdoba, Córdoba, Spain, in 2019. He is currently pursuing the Ph.D. degree in advanced computing technologies with the University of Castilla–La Mancha, Ciudad Real, Spain.

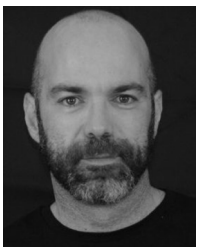
His current research interests include machine learning and the use of IoT technologies, particularly monitoring systems in smart home environments.

Mr. Llumiguano Solano was awarded a scholarship by the Asociación Universitaria Iberoamericana de Postgrado (AUIP) to pursue his master's studies.



**Jesús Fernández-Bermejo Ruiz** received the B.S. degree in computer engineering, the M.Sc. degree in computer engineering, and the Ph.D. degree in advanced computing technologies from the University of Castilla–La Mancha, Ciudad Real, Spain, in 2019, 2020, and 2024, respectively.

He is currently an Assistant Professor at the School of Computer Science, University of Castilla–La Mancha, where he carries out both teaching and research activities, contributing to various academic projects and collaborations. His current research interests include machine learning, sensor networks, artificial intelligence, and the use of IoT technologies for innovative healthcare solutions, particularly in elderly care and smart monitoring systems.



**Félix Jesús Villanueva Molina** received the B.Sc. and Ph.D. degrees in computer engineering from the University of Castilla–La Mancha (UCLM), Ciudad Real, Spain, in 2001 and 2009, respectively.

In 2004 and 2009, he spent semestral research stays at the University of Twente, Enschede, The Netherlands, and at the Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany. He is currently an Associate Professor at UCLM. His research interests include the Internet of Things, ambient intelligence, and embedded systems.



**Fernando Rincón Calle** (Member, IEEE) received the Ph.D. degree in computer engineering from the Autonomous University of Barcelona, Bellaterra, Spain, in September 1993, and the Ph.D. degree in computer engineering from the University of Castilla–La Mancha, Ciudad Real, Spain, in May 2003.

He is currently an Assistant Professor at the University of Castilla–La Mancha. He was the Director of the Department of Information Technology and Systems from 2010 to April 2017. His scientific contributions include more than 20 publications in international impact journals, eight book chapters, two patents, and over 50 contributions to national and international conferences. He has extensive experience in both publicly funded research projects and has participated in over 25 contracts with leading Spanish companies. His current lines of research include edge computing, electronic design automation, and dynamically reconfigurable systems based on FPGA devices.